Chapter16 贪心算法

Ch16.贪心算法

求最优解的问题可看作是通过一系列步骤，每一步有一选择的集合，对于较简单的问题，动态规划显得过于复杂，可用较简单有效的算法求解之。

贪心算法总是在当前步骤上选取最好的方案，即它是一种局部最优的选择，并希望它导致一个全局最优，但有时是不可能导致全局最优。

例：求v到w的一条最短路径，若从v搜索到邻点t最短，未必是v到w最短。

但是，仍有许多问题贪心法将产生全局最优解，如MST，单源最短路径等。

16.1 活动选择问题

- 多个活动竞争资源的调度问题：尽可能多地选择互不冲突的活动
- 设有n个活动$S=\{a_1, a_2,\ldots,a_n\}$，均要使用某资源(如教室)，该资源使用方式为独占式，一次只供一个活动使用
  - 每个活动a_i发生的时间为$[s_i, f_i)$, $0 \leqslant s_i < f_i < \infty$
  - 两活动a_i, a_j相容（不冲突）是指： $[s_i, f_i)$, $[s_j, f_j)$不重叠，满足$s_i \geqslant f_j$ or $s_j \geqslant f_i$。即：一活动的开始时间大于等于另一活动的完成时间。
  - 活动选择问题：选择最多的互不冲突的活动，使相容活动集合最大。即：$A \subseteq S$, A中活动互不冲突且$|A|$最大。

16.1 活动选择问题

- 例（按完成时间递增序排列）

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14

- 问题的解：$A_1=\{a_3,a_9,a_{11}\}$, $A_2=\{a_1,a_4, a_8,a_{11}\}$

  $A_3=\{a_2,a_4,a_9a_{11}\}$

- 最优解： A_2和A_3

此问题可用迭代方法直接给出贪心算法，但为比较和动态规划之关系，下面先考虑动态规划解法。

16.1 活动选择问题

1、活动选择问题的最优子结构

- 子问题空间可描述为：$S_{ij} = \{a_k \in S : f_i \le s_k < f_k \le s_j\}$

S_{ij}是S的子集，它包含那些（可在活动a_i完成之后开始）and（可在活动a_j开始之前完成）的活动a_k。

亦即：S_{ij}中所有活动a_k与活动a_i、a_j相容，不妨称a_i、a_j和子集S_{ij}相容，因此a_k也与所有不迟于f_i完成的活动以及与所有不早于s_j开始的活动相容。Note：S_{ij}中可能有冲突的活动。

为方便处理，设有两个虚拟的活动a_0和a_{n+1}，并且假定 $f_0 = 0, s_{n+1} = \infty$

因此，$S = S_{0,n+1}, 0 \le i, j \le n+1$

16.1 活动选择问题

为了更严格定义i和j的范围，假定所有活动已按完成时间的单调增有序：

$$f_0 < f_1 \le f_2 \le \ldots \le f_n < f_{n+1} \qquad 16.1式$$

$\because S_{ij} = \varphi$,若$i \ge j$ //易于用反证法证明

$\therefore$只考虑S_{ij},当$i < j$。因此i, j的范围是:$0 \le i < j \le n+1$

16.1 活动选择问题

- 如何分解问题？

设子问题 $S_{ij} \neq \phi, a_k \in S_{ij}, f_i \leq s_k < f_k \leq s_j$ 。若S_{ij}的解中选择了a_k，使用a_k产生S_{ij}的两个子问题：

$$\left.\begin{array}{l} S_{ik}: \text{包括在} a_i \text{完成之后开始，在} a_k \text{开始前完成的那些活动} \\ S_{kj}: \text{包括在} a_k \text{完成之后开始，在} a_j \text{开始前完成的那些活动} \end{array}\right\rangle$$

均是S_{ij}的子集，这两子集与a_k相容

S_{ij}的解显然是S_{ik}和S_{kj}解的并，再加上a_k。

注意S_{ik}和S_{kj}已去掉了那些与a_k冲突的活动，而这些活动原来可能在S_{ij}中。

16.1 活动选择问题

❖最优子结构

设S_{ij}的最优解是A_{ij}，$a_k \in A_{ij}$，

则两子问题S_{ik}和S_{kj}的解A_{ik}和A_{kj}也必须是最优的

易用反证法证明：

因为A_{ik}和A_{kj}是独立的，可用$cut-and-paste$证明

❖用子问题的最优解构造原问题的最优解

$$A_{ij} = A_{ik} \cup \{a_k\} \cup A_{kj} \qquad (16.2式)$$

整个问题的最优解是$S_{0,n+1}$的一个最优解

16.1 活动选择问题

2、一个递归解

设C[i,j]表示S_{ij}的最优解的值，即S_{ij}中相容活动最大子集的size：C[i,j]=$|A_{ij}|$

1) 当$S_{ij}=\phi$时，$C[i,j]=0$，此时$i \geq j$ // $\because A_{ij} \subseteq S_{ij}, \therefore A_{ij}=\phi$

2) 当$S_{ij} \neq \Phi$时，假设$a_k \in A_{ij}$，则可用S_{ik}和S_{kj}两子问题的最优解来构造S_{ij}的最优解，由16.2式：

$C[i,j]=C[i,k]+C[k,j]+1, i+1 \leq k \leq j-1$, // k有$j-i-1$选择

$$\left|A_{ij}\right|=\left|A_{ik}\right|+\left|\{a_k\}\right|+\left|A_{kj}\right|=\left|A_{ik}\right|+\left|A_{kj}\right|+1$$

$$C[i,j]=\begin{cases}0 & \text{当} S_{ij}=\Phi \\ \max\limits_{i<k<j}\{C[i,k]+C[k,j]+1\} & \text{当} S_{ij} \neq \Phi\end{cases}$$

16.1 活动选择问题

3、动态规划到贪心法的转化

到此可直接写出动态规划解，留作练习

可通过下面的定理简化其解，该定理也说明贪心算法的正确性。

Th16.1 设S_{ij}是任一非空子问题，a_m是S_{ij}中具有最早完成时间的活动：$f_m=\min\{f_k: a_k \in S_{ij}\}$，则：

①活动a_m必定被包含在S_{ij}的某个最优解中；

②子问题S_{im}是空集，使S_{mj}是唯一可能的非空集

//选a_m的目的

pf：先证第2部分：

假定S_{im}非空，则$\exists a_k \in S_{im}$，使$f_i \le s_k < f_k \le s_m$

$\because a_k$的完成时间先于a_m且$a_k \in S_{ij}$

这与a_m是S_{ij}的最早完成活动矛盾！

再证第1部分：

设A_{ij}是S_{ij}的某个最优解，

并假设A_{ij}中的活动已按完成时间单调增排过序，a_k是其中第一个活动。

若$a_k = a_m$，则问题已得证，即最优解包含a_m

若$a_k \neq a_m$，则构造子集$A_{ij}' = A_{ij} - \{a_k\} \cup \{a_m\}$，

只需证A_{ij}'也是最优解即可

$\because a_k \in A_{ij} \subseteq S_{ij}$

$\therefore f_m \leq f_k$

又$\because a_k$是A_{ij}中最早完成的任务，a_m比a_k更早完成

$\therefore A_{ij}'$中的活动互不冲突，也即A_{ij}'是S_{ij}的一个解

$\because |A_{ij}'| = |A_{ij}|$

$\therefore A_{ij}'$亦是S_{ij}的一个最优解，它包含a_m

16.1 活动选择问题

- 该定理的价值可简化问题的解
  - 在动态规划求解时，原问题S_{ij}可分解为两个子问题S_{ik}和S_{kj}求解，且这种分解有j-i-1中可能
  - 定理16.1将其简化为：
    - ①求S_{ij}的最优解时只用到一个子问题，另一个子问题为空；
    - ②只须考虑一种选择：即选S_{ij}中最早完成的活动
  - 该定理带来的另一个好处是：能以自顶向下的方式解每一个子问题：

16.1 活动选择问题

①对原问题$S=S_{0,n+1}$，选其中最早完成的活动a_{m1}

∵S中的活动已按完成时间单调增有序

∴m1=1。下一子问题应是$S_{m1,n+1}$//已去掉与a_{m1}冲突的活动

②选$S_{m1,n+1}$中最早完成的活动a_{m2}

∵须将S中与a_{m1}冲突的活动删除才能得到$S_{m1,n+1}$

∴$m2$未必为2

③一般形式，当选择a_{mi}到解集中之后，需解的子问题变为$S_{mi,n+1}$

显然，所选活动是按完成时间单调增有序的(m1<m2<…<mi<…)

16.1 活动选择问题

- 贪心选择

当某个a_{mi}加入解集合后，我们总是在剩余的活动中选择第一个不与a_{mi}冲突的活动加入解集合，该活动是能够最早完成且与a_{mi}相容。

这种选择为剩余活动的调度留下了尽可能多的机会。即：留出尽可能多的时间给剩余的尚未调度的活动，以使解集合中包含的活动最多。

每次选一个最早完成并与刚加入解集元素相容的活动

16.1 活动选择问题

4、递归的贪心算法

- 输入：向量s和f，并假定已按f单调增有序
  子问题S_{ij}的下标
- 输出：S_{ij}的最优解
- 算法：

```
RecursiveActivitySelector(s, f, i, j){
  //求S_ij的最优解. 原问题的解： i = 0, j = n + 1
  m ← i + 1; //当i = 0时， 有a_1必加入解集
  while (m < j) and (s_m < f_i) do
      //将与a_i冲突的活动去掉，才能得到S_ij
      m ← m + 1; //在a_{i+1}, a_{i+2}, ..., a_{j-1}中找第1个与a_i相容的活动a_m
  if (m < j) then //s_m ≥ f_i, a_m是S_ij中的第1个活动，
                  //即在a_i完成后开始且最早能完成的活动
      return  {a_m} ∪ RecursiveActivitySelector(s, f, m, j);
      // a_m和S_mj最优解并
  else
      return  Φ; //a_j开始前能够完成的活动均与a_i冲突
}
```

16.1 活动选择问题

- 时间
  - 若要排序，则时间为 $\Theta(n \lg n)$
  - 若已排序，则RecursiveActivitySelector(s,f,0,n+1)的时间为 $\Theta(n)$

    在所有的调用里，每个活动被检查一次，请注意每次循环检查时，活动序号只增加不减少，从RAS(s,f,i,j)调用到RAS(s,f,m,j)时，必有i<m。j始终为n+1

16.1 活动选择问题

5、迭代的贪心算法

上述递归很接近于尾递归，只是结束前多了一个Union操作，很易转换为迭代算法：

①j始终不变，j=n+1

②当一个活动a_i加入解集合之后，我们只要从a_i依次往后找到第一个不与a_i冲突的活动a_m，由于活动事先按完成时间单调增有序，故a_m是最早完成且与a_i相容的活动，它也是S_{ij}中的第一个活动

GreedyActivitySelector(S, f){// f单调增有序

$n \leftarrow length[s]$;

$A \leftarrow \{a_1\}$;// A为解集合

$i \leftarrow 1$;// i是最近加入解集合A的活动a_i的下标

for $m \leftarrow 2$ *to* n *do* //找$S_{i,n+1}$中最早完成的活动a_m

if $(f_i \le s_m)$*then*{// $i < m, a_m$与a_i相容

$A \leftarrow A \cup \{a_m\}$;// a_m是与a_i相容且完成时间

//最早的活动

$i \leftarrow m$;// i仍记录最近加入A的活动a_i的下标

}//endif

return A;

}

16.2 贪心策略要点

- 贪心算法的一般设计步骤

  ① 将优化问题分解为做出一种选择及留下一个待解的子问题

  ② 证明对于原问题总是存在一个最优解会做出贪心选择，从而保证贪心选择是安全的

  ③ 验证当做出贪心选择之后，它和剩余的一个子问题的最优解组合在一起，构成了原问题的最优解

- 没有一个一般的方法告诉我们一个贪心算法是否会解决一个特殊的最优问题，但是有两个要素有助于使用贪心算法：

  贪心选择性质，　最优子结构

16.2 贪心策略要点

1、贪心选择性质

- 贪心选择性质：一个全局最优解能够通过局部最优（贪心）选择达到
- 贪心法总是在当前步骤上选择最优决策，然后解由此产生的子问题
- 贪心选择只依赖了目前所做的选择，但不依赖于将来的选择及子问题的解
- 自顶向下，每做一次贪心选择，就将子问题变得更小

16.2 贪心策略要点

2、最优子结构

- 和动态规划一样，该性质也是贪心法的一个关键要素

  例如，活动选择问题的动态规划解中：

S_{ij}的最优解 $A_{ij} \Rightarrow$ 若 $a_k \in A_{ij}$，则 A_{ij} 包含 S_{ik} 和 S_{kj} 的最优解

- 对贪心算法更直接

  原问题的最优解 $\Rightarrow$ 贪心选择 + 子问题的最优解

  在贪心算法中使用最优子结构时，通常是用更直接的方式。假设在原问题中做了一个贪心选择得到一个子问题，对子问题可使用归纳法证明在每一步上做贪心选择可产生最优解。

16.2 贪心策略要点

3、贪心法与动态规划的比较

- 相同点：两种方法都利用了最优子结构特征
- 易错误处：
  ① 当贪心算法满足全局最优时，可能我们试图使用动态规划求解，但前者更有效
  ② 当实事上只有动态规划才能求解时，错误地使用了贪心法

为了说明两种技术的细微区别，请看一个古典优化问题的两个变种：

背包问题：n个物品重$w_1,w_2,\ldots,w_n$,背包可放入重量W，问能否从n件物品中选择若干件放入背包，使重量和正好等于W。

16.2 贪心策略要点

- **0-1背包问题和零头背包问题**

物品件数：n；第i件物品价值\$$v_i$，重量$w_i$磅，

v_i和w_i为整数，背包载重量：W磅（整数）

怎样选物品装包使得包内含有价值最大，且总重量≤W

不允许一物品多次装包，可理解为n个物品互不相同

- **0-1背包：某物品拿与不拿(1,0的选择)**
- **零头背包：某物品可取部分装包**

想象：小偷偷东西，包容量有限，想尽可能使偷走的东西总价值最大，0-1背包偷走的是金条之类物品，零头背包偷走的是金粉之类物品。

16.2 贪心策略要点

- 两个背包问题都具有最优子结构
  - 0-1背包问题

    原问题的最优解：设背包中装有重量至多为W磅的最有价值的若干物品；

    子问题的最优解：从原问题的最优解中去掉某物品j，则包中剩余物品应是除j外，取自原n-1件物品中最有价值，且总重不超过$W-w_j$的若干物品。

    显然，原问题分解为子问题时，原问题的最优解也要求子问题的解最优

16.2 贪心策略要点

- 零头背包

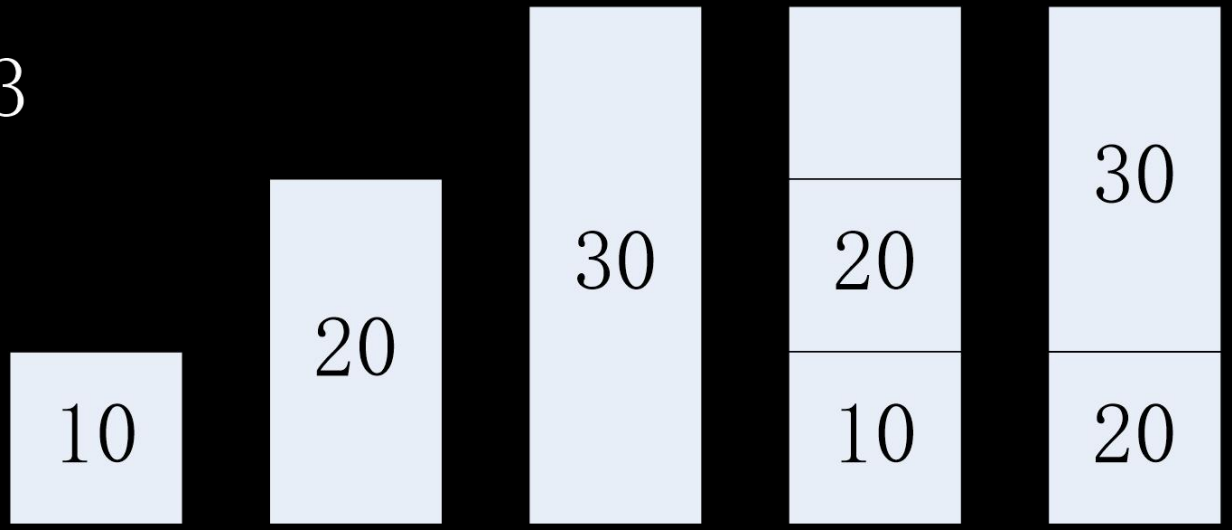
  子问题：从背包中去掉物品j，重量为w(该物品剩余w_j-w)，则包中剩余物品应是除j之外，取自原n-1件物品以及物品j的w_j-w部分中最有价值且总重至多为W-w的若干物品

  显然，原问题解最优亦要求子问题的解最优

16.2 贪心策略要点

- 两个背包问题的不同解法
  - 0-1背包只能用动态规划求解
    - 按每磅价值(v_i/w_i)排序
    - 贪心选择策略：取单位价值最大者装包，若装不下，考虑下一单位价值最大的物品，直至包装满或所有物品都考虑过为止
    - 实际上，装入当前每磅价值最大者只能保证当前最优（局部最优），然而放弃它可能使得后续选择更优。所以在装包前，应将某物品装包的子问题的解和放弃它的子问题的解进行比较，这将导致许多重叠子问题，这正是动态规划的特点。

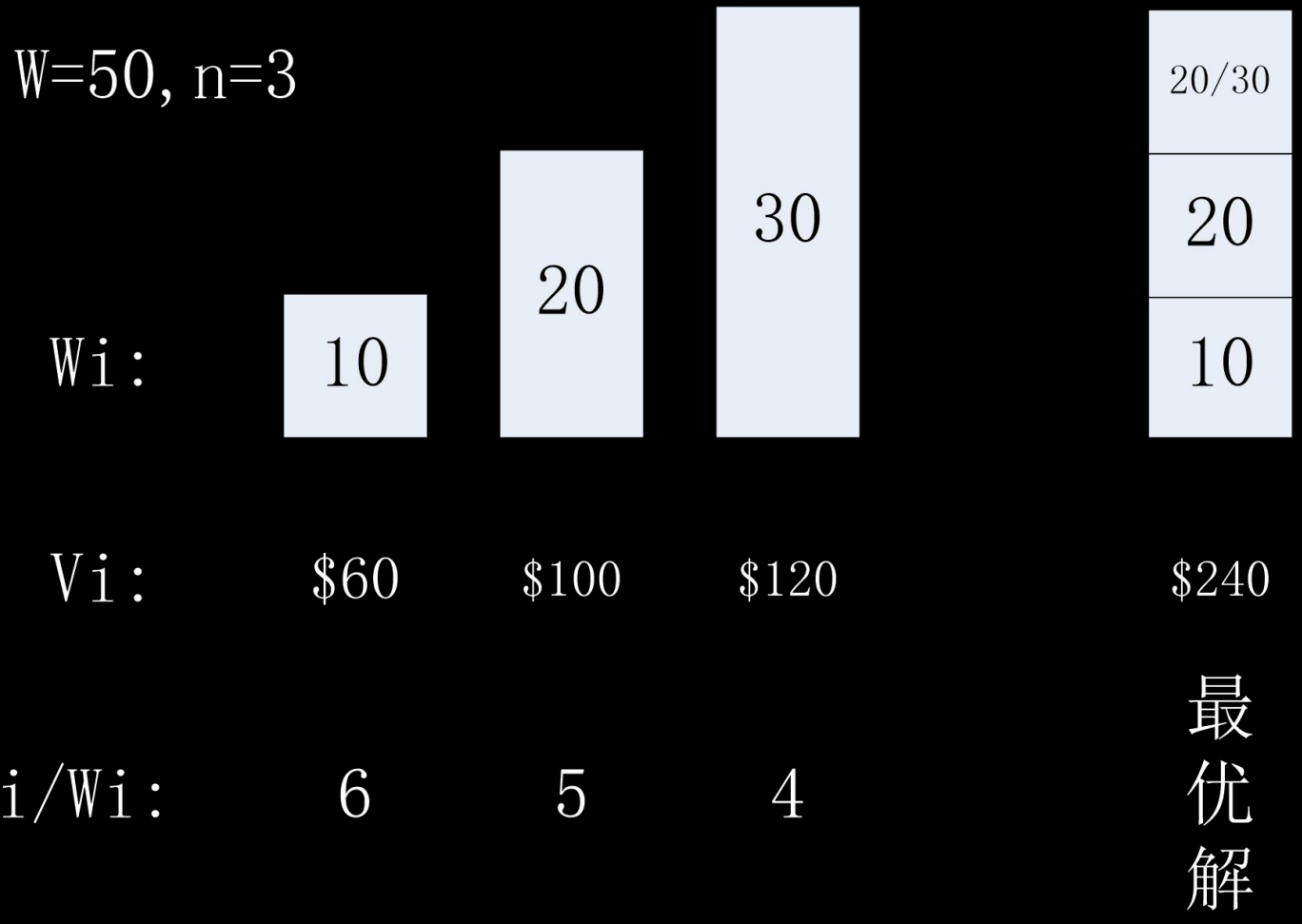
W=50, n=3



Vi:	$60	$100	$120	$160	$220
Vi/Wi:	6	5	4	解	最优解

16.2 贪心策略要点

- 零头背包：可用贪心算法求出最优解



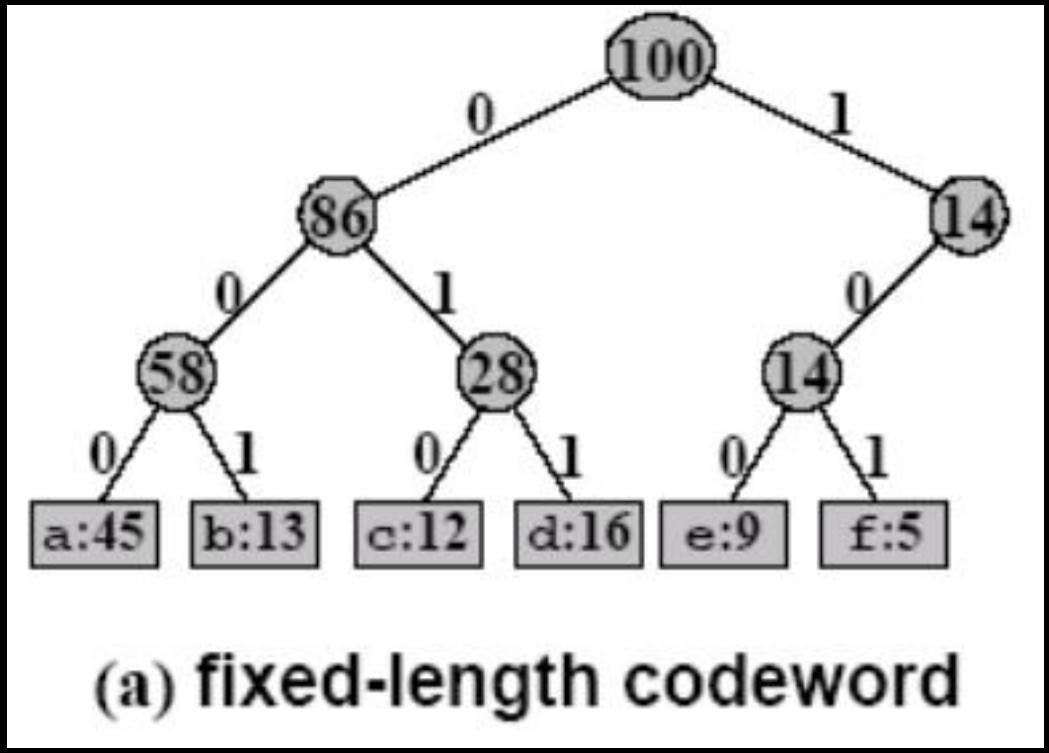
16.3 Huffman编码

- 假设有一个包含100000个字符的数据文件要存储，各字符在文件中的出现频度见下图。文件中仅有6种不同字符出现过。

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

- 采用二进制字符编码表示每个字符。如果采用固定长度编码，需要三位二进制数字来表示六个字符。这种方法需要300K位对整个文件编码。
- 可变长编码需要 45*1 + 13*3 + 12*3 + 16*3 + 9*4 + 5*4 = 224K 位，可压缩25%

- 解码过程可以解释为从根至叶子（字符）的路径，其中0表示转向左子结点，1表示转向右子结点



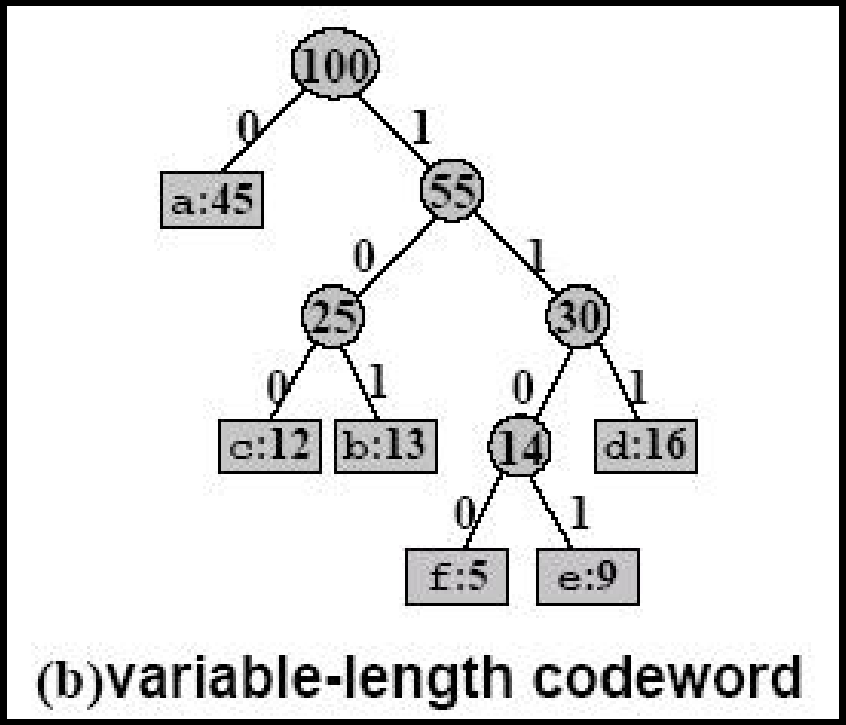
前缀编码

- 最优字符编码可以用某种前缀编码来获得。
- 没有一个编码是另一个编码的前缀，这样的编码称为前缀编码。

a	b	c	d	e	f
0	101	100	111	1101	1100

对字母表C中的每个字符c，设f(c)表示c在文件中出现的频度，d(c)表示c的叶子在树中的深度，也是字符c编码的长度。这样，编码一个文件所需要的位数是

$$B(T) = \sum_{c \in C} f(c) d_T(c)$$



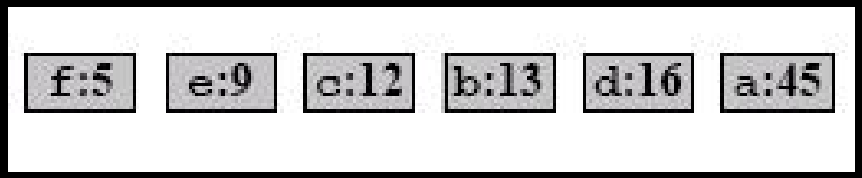
(b)variable-length codeword

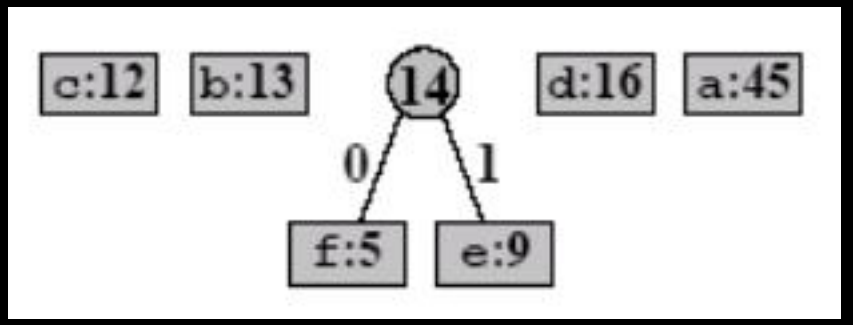
Algorithm of Huffman Coding

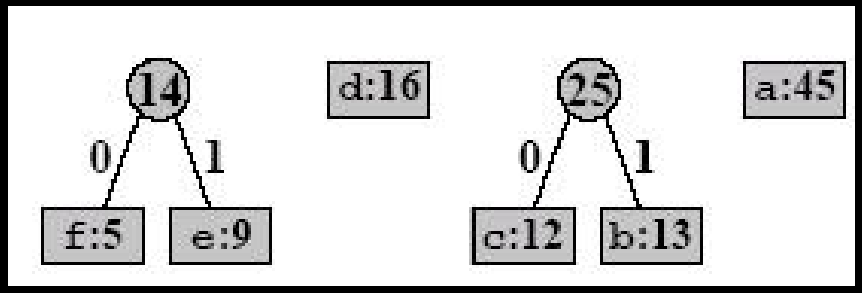
HUFFMAN CODE

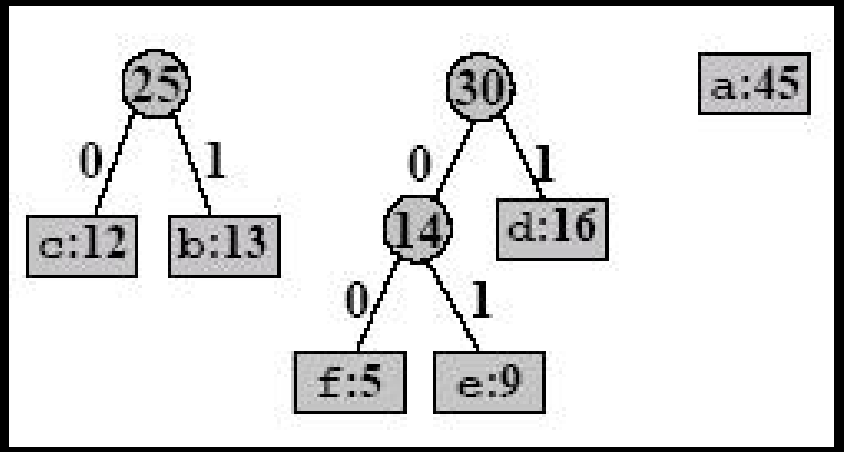
```
HUFFMAN()
1 n ←|C|
2 Q ← C
3 for i ← 1 to n -1
4     do allocate a new node z
5        left-son[z] ← x ← EXTRACT-MIN(Q)
6        right-son[z] ← y ← EXTRACT-MIN(Q)
7        f[z] ← f[x] + f[y]
8        INSERT(Q, z)
9 return EXTRACT-MIN(Q)        //Return the root of the tree
```

Example of Huffman codes



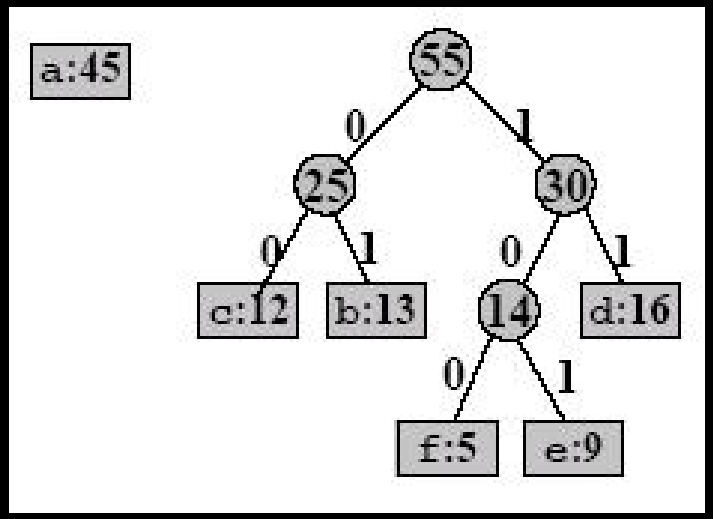


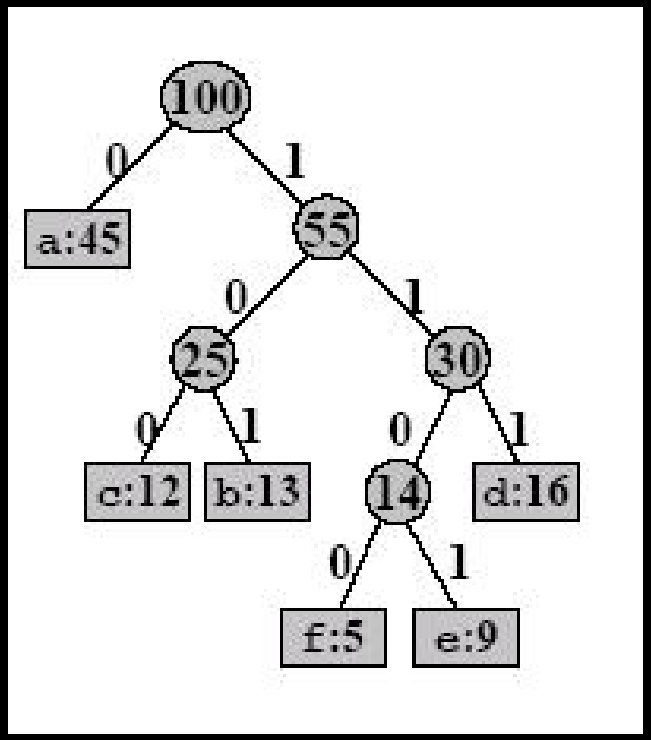




Example of Huffman codes

- (cont.)





16.3 Huffman编码

贪心选择性

贪心策略：Huffman编码构造一棵最优树的过程是从合并两个频度最低的字符的贪心选择开始。

证明这种贪心策略可以产生一个全局最优解。

引理 16.2 设 C 为一字母表，其中每个字符 $c \in C$ 具有频度 $f[c]$。设 x 和 y 为 C 中具有最低频度的两个字符，则存在 C 的一种最优前缀编码，其中 x 和 y 的编码长度相同但最后一位不同。

引理说明了字符x和y是最优前缀编码中具有最大深度的兄弟叶结点。

16.3 Huffman编码

最优子结构

引理 16.3 设 C 为一给定字母表，其中每个字母 $c \in C$ 都定义有频度 $f[c]$。设 x 和 y 是 C 中具有最低频度的两个字母。并设 C' 为字母表移去 x 和 y，再加上(新)字符 z 后的字母表，亦即 $C' = C - \{x, y\} \cup \{z\}$；如 C 一样为 C' 定义 f，其中 $f[z] = f[x] + f[y]$。设 T' 为表示字母表 C' 上最优前缀编码的任意一棵树。那么，将 T' 中的叶子结点 z 替换成具有 x 和 y 孩子的内部结点所得到的树 T，表示字母表 C 上的一个最优前缀编码。

找钱问题

- 问题定义：

使用2角5分，1角，5分和1分四种面值的硬币时（各种硬币数量不限），设计一个找A分钱的贪心算法，并证明算法能产生一个最优解。设货币种类$P=\{p_1,p_2,p_3,p_4\}$，d_i和x_i分别是p_i的货币单位和选择数量，问题的形式描述为：

$$\min\left\{\sum_{i=1}^{4} x_i\right\}$$

$$s.t.\ \begin{cases} \sum_{i=1}^{4} d_i x_i = A \\ x_i \text{为非负整数} \quad 1 \le i \le 4 \end{cases}$$

找钱问题

- 贪心策略

$$x_1 = \lfloor A/d_1 \rfloor \qquad x_3 = \lfloor (A - d_1x_1 - d_2x_2)/d_3 \rfloor$$

$$x_2 = \lfloor (A - d_1x_1)/d_2 \rfloor \qquad x_4 = A - d_1x_1 - d_2x_2 - d_3x_3$$

- 算法

```
GreedyChange( d[], x[], A)
{//输出x[1..4]
    d[1]=25, d[2]=10, d[3]=5, d[4]=1;
    for i=1 to 3 do
    {   x[i]=A/d[i]; A-=x[i]*d[i];
     }
    x[4]=A;
 }
```

找钱问题

- 贪心选择性质证明：

设$X=(x_1,x_2,x_3,x_4)$是贪心解，$Y=(y_1,y_2,y_3,y_4)$是最优解.

可以证明：$x_1=y_1$

如果$x_1<y_1$，由$x_1=\lfloor A/d_1\rfloor \Rightarrow x_1\le A/d_1<x_1+1\Rightarrow d_1x_1\le A<d_1(x_1+1)$；因此，$\sum_{i=1}^{4}d_iy_i\ge d_1y_1\ge d_1(x_1+1)>A$，产生矛盾；

如果$x_1>y_1$，则$10y_2+5y_3+1y_4\ge 25$（否则，$25y_1+10y_2+5y_3+1y_4<25(y_1+1)\le 25x_1+10x_2+5x_3+1x_4=A$，矛盾），因此用一个25分的硬币替代等值的低于25分的硬币若干个（至少≥3），于是y不是最优解；

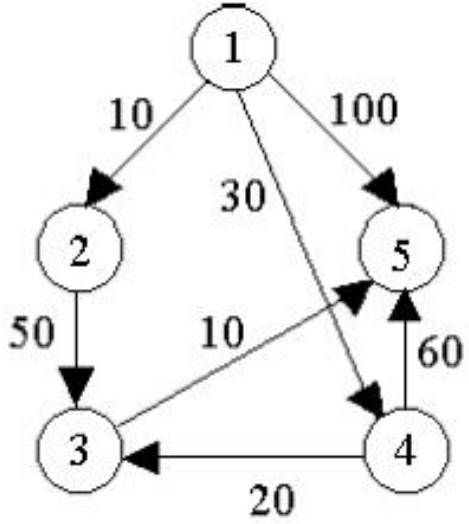
综上，只能$x_1=y_1$；类似可以继续证得$x_2=y_2$，$x_3=y_3$，$x_4=y_4$；

∴X是最优解

单源最短路径

- 问题描述

给定带权有向图G＝(V,E)，其中每条边的权是非负实数。另外，还给定V中的一个顶点，称为源。现在要计算从源到所有其它各顶点的最短路长度。这里路的长度是指路上各边权之和。这个问题通常称为单源最短路径问题。



如：计算顶点1（源）到所有其他顶点之间的最短路径。

单源最短路径

- 迪杰斯特拉(Dijkstra)算法：

其基本思想是：设置顶点集合S并不断地作贪心选择来扩充这个集合。一个顶点属于集合S当且仅当从源到该顶点的最短路径长度已知。

- 步骤：

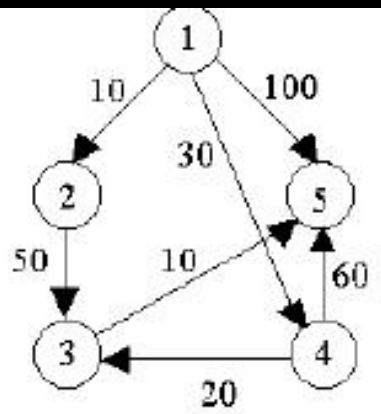
① 初始时，S中仅含有源。设u是G的某一个顶点，把从源到u且中间只经过S中顶点的路称为从源到u的特殊路径，并用数组dist记录当前每个顶点所对应的最短特殊路径长度。

② 每次从V-S中取出具有最短特殊路长度的顶点u（贪心策略），将u添加到S中，同时对数组dist作必要的修改。

③ 知道S包含了所有V中顶点，此时，dist就记录了从源到所有其它顶点之间的最短路径长度。

单源最短路径

例如，对右图中的有向图，应用迪杰斯特拉算法计算从源顶点1到其它顶点间最短路径的过程列如下表所示。



迭代	S	u	dist[2]	dist[3]	dist[4]	dist[5]
初始	{1}	-	10	maxint	30	100
1	{1,2}	2	10	60	30	100
2	{1,2,4}	4	10	50	30	90
3	{1,2,4,3}	3	10	50	30	60
4	{1,2,4,3,5}	5	10	50	30	60

单源最短路径

- 算法描述如下：

```
Dijkstra( int n, int v, Type dist[], int prev[], Type ** c)
{ // 单源最短路径问题的迪杰斯特拉算法，
    bool  s[maxint];
    for ( int i = 1; i<=n;i++) {
        dist[i]=c[v][i];
        s[i]= false;
        if ( dist[i]==maxint) prev[i] = 0;
        else prev[i] = v;
    }
    dist[v]=0; s[v]=true;
    for ( int i=1; i<n; i++) {
        int temp = maxint;
        int u = v;
        for ( int j=1; j<=n; j++ )
            if ( (!s[j])&&(dist[j]<temp)) { u=j; temp = dist[j]; }
        s[u] = true;
        for( int  j=1; j<=n; j++ )
            if ( (!s[j])&&(c[u][j] < maxint)) {
                Type  newdist = dist[u] + c[u][j];
                if ( newdist < dist[j] )  {dist[j]=newdist;prev[j]=u;}
            }
    }
}
```

其中：

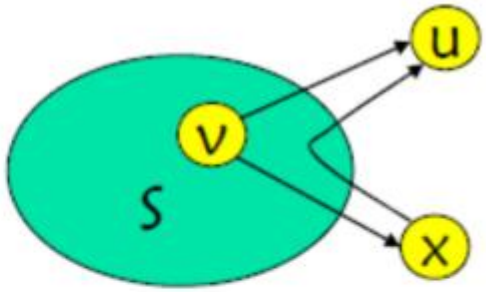
c[i][j]表示边(i,j)的权，n是顶点个数，v表示源，dist[i]表示当前从源到顶点i的最短特殊路径长度

思考：本算法只是给出了从源顶点到其他顶点间的最短路径长度，并没有记录相应的最短路径。该如何修改才可以记录相应的最短路径？

单源最短路径

- 贪心策略为：从V-S中选择具有最短特殊路径的顶点u，

从而确定从源到u的最短路径长度dist[u]。



- 贪心选择性质证明：

证明：（反证法）

即证明从源到u没有更短的其他路径。

假设存在一条从源到u且长度比dist[u]更短的路，设这条路初次走出S之外到达顶点为x#V-S，然后徘徊于S内外若干次，最后离开达到u，如上图所示。

在这条路径上，分别记d(v,x)，d(x,u)和d(v,u)为顶点v到顶点x，顶点x到顶点u和顶点v到顶点u的路长，那么

$$dist[x] <= dist(v,x) \quad d(v, x) + d(x, u) = d(v,u) < dist[u]$$

利用边权的非负性，可知d(x,u)>=0，从而推得dist[x]<dist[u]。此为矛盾。这就证明了dist[u]是从源到顶点u的最短路径长度。

16.4 贪心算法的理论基础

该理论可用来确定贪心法何时能产生最优解，它用到了一种组合结构：matroid（胚）。该结构是Whitney于1935年在研究矩阵中线性无关结构时抽象出来的，有Korte于80年代初将该理论发展为贪心算法理论。该理论覆盖了许多贪心法的应用实例，但并未覆盖所有情况（如活动选择问题），但它仍在发展。

16.4.1 胚

- Def：一个胚是满足下列条件的有序对M=(S,I)
  - ① S是一有穷非空集
  - ② I是S的一个非空子集（称为S的独立子集）簇，使得若B∈I且A⊆B，则A ∈ I。满足此性质的I是遗传的，即B独立（指B∈I），则B的子集也独立。注意Φ∈I。
  - ③ 若A∈I，B∈I且|A|<|B|，则存在某个元素x∈B-A使得A∪{x} ∈ I。称M满足交换性质。

16.4.1胚

- 例子
  - 惠特尼35年研究矩阵胚时引入。一个矩阵胚是指：S的元素是矩阵的行，若行的子集线性无关则该子集独立，易证这种结构是胚。
  - 图的胚

    $M_G=(S_G,I_G)$是在无向图$G=(V,E)$基础上定义的：

    - $S_G=E$
    - 若$A\subseteq E$，则：$A\in I_G \Leftrightarrow A$无回路。

    即：边集A独立 $\Leftrightarrow$ 子图$G_A=(V,A)$是森林

    M_G与MST问题紧密相关

16.4.1胚

- **Th16.5 若G是无向图，则$M_G=(S_G,I_G)$是一个胚**

pf：

1. $S_G = E$是一有穷（非空）集
2. 证I_G是遗传的

   若$B \in I_G$，且$A \subseteq B$，意味着A是森林B的子集，它仍为森林不可能有回路，所以$A \in I_G$。
3. M_G的交换性

   设$G_A = (V, A)$和$G_B = (V, B)$为森林，且$|B| > |A|$，B包含的边多于A。现要证$\exists e \in B - A$，使e扩充到A后仍不产生回路

可利用*Th*B.2：具有k条边的森林包含$|V|-k$棵树

由该定理可知，G_A有$|V|-|A|$棵树，G_B有$|V|-|B|$棵树

$\because G_B$中树的数目$<G_A$中的树的数目

$\therefore G_B$中必存在某棵树T，它的顶点属于森林G_A中两棵不同的树（否则若G_B中任何树的顶点必在G_A的同一棵树中，这与G_A中树数目大于G_B矛盾）

$\because T$是连通的，必存在一条边$e=(u,v)\in T$，使得u,v在G_A的两不同树中

$\therefore$将e加入G_A后不会产生回路，即$A\cup\{e\}\in I_G$，满足交换性。

综合1,2,3知M_G是一个胚

16.4.1胚

- 最大独立集
  - 独立子集的扩张

    在胚M=(S,I)中，若$A \in I$，$x \notin A$，$A \cup \{x\} \in I$，则元素x称为A的一个扩张，即元素x扩充到独立子集A后仍保持独立性

    例：在图的胚M_G中，若A是一个独立子集，则e是A的扩张是指加入e后仍不产生环
  - 胚的最大独立子集

    若A是胚M的独立子集，且无任何扩张，则A称为M的最大独立子集，即在M中没有更大的独立子集能包含A

16.4.1胚

- **Th17.6 胚中所有最大独立子集的大小相同**

pf :[反证法]

设A是M的最大独立子集，且存在另一更大的最大独立子集B。

由交换性知：$\exists x \in B - A$，使$A \cup \{x\} \in I$，

即A是可扩张的，与A是最大独立子集矛盾。

例：在M_G中，每个最大独立子集是一棵生成树，有|v|-1条边

16.4.1胚

- 加权胚

若对∀x∈S，为x指派一个正的权值w(x)，则称M=(S,I)是加权胚。S的子集的权可定义为：

$$w(A)=\sum_{x\in A} w(x) \quad \text{for any } A \subseteq S$$

例：M_G中，可定义w(e)为边e的长度，w(A)为A中所有边的长度之和

16.4.2 加权胚上的贪心算法

- 加权胚的最大独立子集可描述某些问题的最优解（加权胚的最优子集）

可用贪心算法求出最优解的许多问题（但不是全部）可形式化为一加权胚中找到一个权值最大的独立子集。即：给定加权胚M=(S, I)，希望找到I的一个独立子集A，使w(A)最大。

- 胚的最优子集：胚中权值最大的独立子集

  它也一定是胚中的一个最大独立子集（子集体积最大），反之不然。

Pf: ∵∀x∈S，w(x)>0，∴若权最大的独立子集A不是最大独立子集，可将其扩张至最大独立子集，后者的权更大，使A的权非最大

16.4.2 加权胚上的贪心算法

❖**例** 求连通无向图$G=(V,E)$的MST问题$\Rightarrow$加权胚中求权最大的独立子集问题。即：求胚的最优子集

设G的权函数w定义为边的长度，且$w(e)>0$

加权胚M_G的权w'定义为：$w'(e)=w_0-w(e)$，w_0大于各边的最大长度。在M_G中，$\forall e\in E$，$w'(e)>0$,故M_G的每个最优子集A是G的一棵MST

pf：

$\because A$是胚的最大独立子集

$\therefore$它是一棵生成树，它的权为$w'(A)=(|V|-1)w_0-w(A)$

$\because A$也是权最大的独立子集

$\therefore w'(A)$最大必有$w(A)$最小，即A是G的一棵MST

16.4.2 加权胚上的贪心算法

- 加权胚的贪心算法
  - 适用于任何加权胚求最优子集A
  - 贪心之处：尽可能选权值最大的元素扩充到A

```
Greedy(M, w){
//输入胚M = (S, I)，w表示正的权函数
    A ← φ;
    按权w单调递减对S[M]排序
    for 每个x ∈ S[M]，按w(x)单调递减 do
        if (A ∪ {x} ∈ I[M]) then
            A ← A ∪ {x}; //扩充x未破坏A的独立性
        //否则放弃x
    return A;
}
```

16.4.2 加权胚上的贪心算法

时间分析：

设$|S| = n$，排序$O(n \lg n)$

for循环n次，

每次检验$A \cup \{x\}$是否独立，

设检查时间为$O(f(n))$,

总时间为$O(n \lg n + nf(n))$

16.4.2 加权胚上的贪心算法

- **Greedy算法返回1个最优子集**

  A是独立子集易从Φ独立开始用归纳法证明

  - **Lemma16.7 （胚呈现贪心选择性质）**

    设M=(S,I)是加权胚，权函数为w，且S已按权值的单调递减有序，设x是S的第一个使{x}独立的元素（若这样的x存在）。若x存在，则存在S的一个最优子集A包含x。（即说明第一步贪心选择正确）

pf：

(1)若无这样的x存在，则唯一的独立子集是Φ

(2)否则，设B为S的任一非空最优子集

若$x \in B$，则令$A = B$，证毕；

16.4.2 加权胚上的贪心算法

若$x \notin B$，则可从B构造一个最优子集A，

使其包含x，为此需证$w(A) \geq w(B)$

i) 先证$\forall y \in B$，有$w(x) \geq w(y)$

$\because y \in B \quad \therefore \{y\} \subseteq B$

由于B是S的最优子集，故B也是独立子集，

由I的遗传性知$\{y\}$亦是独立子集

$\because \{x\}$是S的第一个独立子集，

而S中元素已按单调减有序，

$\therefore w(x) \geq w(y)$

ii)构造集A，使其包含x且A最优

开始令$A=\{x\}$,显然A是独立，

利用交换性质，重复地在B中找新的元素扩充到A使A独立，直至$|A|=|B|$。于是有：

$A=B-\{y\}\cup\{x\}$ 对某个y成立

由*i*)立即知道：

$$w(A)=w(B)-w(y)+w(x)\geq w(B)$$

由B是最优子集即可知A亦是最优子集，且它包括x。

16.4.2 加权胚上的贪心算法

下面的引理和推论说明：若一元素开始未被选中，则此后亦不可能被选中

- **Lemma16.8 设M=(S,I)是任一胚，若S的一个元素x是S的某个独立子集A的一个扩张，则x亦是Φ的一个扩张**

$pf:$

$\because x$是A的扩张

$\therefore A \cup \{x\} \in I$

由I的遗传性知，$\{x\}$是独立的，它是ϕ的一个扩张

16.4.2 加权胚上的贪心算法

- 推论16.9 设M=(S,I)是任一胚，若S的一个元素x不是Φ的扩张，则x也不是S的任何独立子集A的一个扩张

  pf：由引理16.8易证

- 该推论告诉我们，若一开始某元素没被选中，此后亦不会选中，保证Greedy算法开始的正确性

16.4.2 加权胚上的贪心算法

- 引理16.10（胚具有最优子结构性质）

对于加权胚M=(S,I)，设x是Greedy算法选中的第一个元素，找一个包含x的权最大的独立子集的剩余问题可归结为找加权胚M’=(S’,I’)的一个权值最大的独立子集，这里：

$$S' = \{y \in S : \{x, y\} \in I\}$$

//即S'由S中可扩张至$\{x\}$中的元素构成

$$I' = \{B \subseteq S - \{x\} : B \cup \{x\} \in I\}$$

//即I'是由S的不包含x的独立子集构成

且M’的权函数是M的权函数，但只限于S’。我们称M’是由元素x引起的M的收缩，它是M的子问题。

pf：

若A是M的任一包含x的最优子集（即权最大的独立子集），则由I'的定义可知：

$A' = A - \{x\}$是M'的一个独立子集，

反之，M'的任一独立子集A'产生M的一个独立子集：$A = A' \cup \{x\}$

∵ 两种情况下，均有$w(A) = w(A') + w(x)$

∴ 包含x的A是M中权最大的独立子集，保证了$w(A')$须最大，即A'是M'的最优子集，反之亦然

即：A是原问题M的最优解要求A'是子问题M'的最优解；反之，$\{x\} \cup A'$构成原问题的最优解A

16.4.2 加权胚上的贪心算法

- **Th16.11 （胚的贪心算法的正确性）**

  若 M=(S,I) 是权函数为 w 的加权胚，则调用 Greedy(M,w)将返回一个最优子集

pf:

①由推论16.9知，一开始被忽略的元素可以放弃，因为它们不是Φ的扩张意味着此后也不会是任何独立子集的扩张。

②一旦一个元素x被选中，引理16.7保证了算法将其扩充到A中是正确的，因为存在一个最优子集包含x。

16.4.2 加权胚上的贪心算法

③引理16.10蕴含着剩余子问题是在M’中找一最优子集。在Greedy将A置为{x}之后，剩下的各步骤可解释为是在胚M’中进行的，因为∀B∈I’，B在M’中独立等价于B∪{x}在M中独立。因此，Greedy的后续操作将找出M’的一个最优子集（可用归纳法），Greedy的全部操作可以找到M的最优子集

- 若一应用问题能抽象为加权胚，则一定能用贪心法求出其最优解

16.5 一个任务调度问题

- 问题描述

在单处理器上对单位时间任务进行最优调度

- 输入
  - 任务集：$S=\{a_1,a_2,\ldots,a_n\}$，每个任务在单位时间内完成，总时间为n
  - 截止期：$\forall a_i \in S$，a_i的截止期d_i为整数，且$1 \le d_i \le n$，即a_i须在d_i或d_i时刻之前完成
  - 权（罚款）：$\forall a_i \in S$，a_i的权$w_i \ge 0$，表示a_i没有在d_i或d_i之前完成所导致的罚款，但提前完成或按时完成的任务没有罚款
- 输出：求出S的一个调度，使总罚款最小

显然S的任意枚举都是一个调度方案，在总时间n内肯定完成，但我们力图使延期完成的任务权值和最小

16.5 一个任务调度问题

- 将问题抽象为加权胚
  - 早任务：在给定的调度中，能按期或提前完成的任务（即在deadline之前完成任务）
  - 迟任务：在给定的调度中，在截止期之后完成的任务
  - 早任务优先形式：将早任务排在迟任务之前的调度
  - 任何调度均可安排成早任务优先形式

例：……a_j……a_i……　→　……a_i……a_j……

迟　早　　早　迟

交换位置不影响a_i和a_j的早迟特性，即总罚款不变

16.5 一个任务调度问题

- 调度的规范形式

  早任务在前、迟任务在后（即先将其按早任务优先形式调度）；并且将所有早任务按deadline单调增次序调度（迟任务可按任意次序调度）。

- 规范形式的调度不改变任务的早迟特性，因此任何调度均可安排成该形式而保持总罚款不变

16.5 一个任务调度问题

- 最优调度的规范次序

因为任一调度都有一规范形式，所以最优调度也存在规范形式

①在最优调度S中找出早任务集A

②将A中任务按deadlines递增序排列

③迟任务集S-A可按任意序排列

注意：

因为早任务不导致罚款，迟任务的次序也不改变罚款的多少（权的大小），所以上述安排不改变最优调度的权值

16.5 一个任务调度问题

- 独立任务集

若存在一个调度使得任务集A中没有迟任务，则称A是独立的。

下面的引理可判定一给定任务集A是否独立，为此先给出一个定义。

- Def： 对 t=1,2,…,n, 设 $N_t(A)$ 表示集合 A 中deadline小于等于t的任务数目， $N_0(A)=0$ for any A。

  即$N_t(A)$表示A中想要在t及t时刻之前完成的任务个数，显然$N_t(A)\le|A|$，$0\le t\le n$

16.5 一个任务调度问题

- 引理16.12 对任意的任务集，下列命题等价：

  (1)集合A独立

  (2)对t=0,1,…,n，有$N_t(A) \le t$

  (3)若对A中的任务按deadline单调增序进行调度，则A中无迟任务

Pf：(1)→(2) (反证法)

若(1)成立，则存在一个调度，使A中无迟任务

若存在t使$N_t(A)>t$，则A中有多于t个任务要在t及t之前完成，因此找不到一个调度使A中任务均在t及t之前完成，即任何调度均使A中至少有一个迟任务，与A独立矛盾。

16.5 一个任务调度问题

(2) →(3)

实际上只要 “1≤t≤|A|，有$N_t(A) \le t$” 即可

因为t>|A|时， $N_t(A) \le |A| < t$

若(2)成立，说明A是某个调度的早任务集，他的规范次序不会改变A的早任务性质。

∵ ∀i∈[0,n]，在i及i之前要完成的任务至多为i个

∴对A中任务按deadline增序调度时A中无迟任务

(3) →(1)

按(3)调度A中无迟任务，所以A独立

16.5 一个任务调度问题

- 使迟任务罚款和最小等价于使早任务罚款和最大（即权最大的独立子集）

  下面的定理保证贪心法可求出最大总罚款的独立任务集A，即把该问题抽象为加权胚，从而由Th16.11保证贪心法可求出具有最大权值的独立任务集A

- Th16.13 若S是一个具有deadlines的单位时间任务的集合，且I为所有独立任务集构成的集合，则相应系统(S,I)是一个胚。

  Pf:(1) S有穷，非空

  (2) I的遗传性：∵独立的任务集是指对某调度而言没有迟任务的集合，∴独立任务集的任一子集必是独立的

16.5 一个任务调度问题

(3)M的交换性

设A、B∈I且|B|>|A|，设k是使$N_t(B)\le N_t(A)$成立的最大的t.

$N_n(B)=|B|$，$N_n(A)=|A|$（注意|S|=n，$|A|\le n$，$|B|\le n$，$1\le d_i\le n$，在n前所有任务可完成）

但是|B|>|A|⇨ $N_n(B)>N_n(A)$，所以k≠n，即k<n

①当t≤k时有： $N_t(B)\le N_t(A)$（这样t一定存在，因为$N_0(B)=N_0(A)=0$）

②当k+1 ≤j ≤n有$N_j(B)>N_j(A)$（注意$N_t(B)$和$N_t(A)$均为单调增for t ）

16.5 一个任务调度问题

∴.B中deadline为k+1的任务多于A，故可设$a_i \in B-A$且$d_i=k+1$.

设A’=A∪{a_i}，只要证A’独立即可

下面用引理16.12性质2来证A’独立，即要证

$$\forall t \in [0,n], 有 N_t(A') \le t$$

①对1≤t≤k

显然$N_t(A')=N_t(A)$（因为A’ 中的a_i截止期为k+1）

又因为A独立，由引理16.12性质2知：

$N_t(A')= N_t(A) \le t$，$1 \le t \le k$ <式1>

16.5 一个任务调度问题

②对k+1≤t≤n

∵ $N_t(A)<N_t(B)$, $|A'|=|A|+1$

∴ $N_t(A') \le N_t(B)$

又∵ B独立，由引理16.12性质2知：

$N_t(A') \le N_t(B) \le t$， k+1≤t≤n。 <式2>

综合<式1>和<式2>可得：

对1≤t≤n，有$N_t(A') \le t$

由性质2知A’独立

16.5 一个任务调度问题

- 利用加权胚的Greedy算法求最优调度
  - 通过Th16.11，可使用Greedy算法找权最大的独立任务集A，A是早任务集。
  - 时间

$$T(n) = O(n \lg n + nf(n))$$
$$= O(n \lg n + n \cdot n) = O(n^2)$$

注意到独立性检查时间为O(n)（实际上可减小到O(|A|)）

16.5 一个任务调度问题

- 例子

a_i	1	2	3	4	5	6	7
d_i	4	2	4	3	1	4	6
w_i	70	60	50	40	30	20	10

贪心法求出的最优子集A={1,2,3,4,7},S-A={5,6}（迟任务集）

规范形式的最优调度<a_2,a_4,a_1,a_3,a_7,a_5,a_6>

总罚款$w_5+w_6=50$

课内练习16.1

1、磁带最优存储问题

★问题描述：

设有 n 个程序$\{1,2,\cdots,n\}$要存放在长度为 L 的磁带上。程序 i 存放在磁带上的长度是 $l_i,1\leqslant i\leqslant n$。这 n 个程序的读取概率分别是 $p_1,p_2,\cdots,p_n$，且 $\sum_{i=1}^{n} p_i=1$。如果将这 n 个程序按 $i_1,i_2,\cdots,i_n$ 的次序存放，则读取程序 i_r 所需的时间 $t_r=c\sum_{k=1}^{r} p_k l_{ik}$。这 n 个程序的平均读取时间为 $\sum_{r=1}^{n} t_r$。

磁带最优存储问题要求确定这 n 个程序在磁带上的一个存储次序，使平均读取时间达到最小。试设计一个解此问题的算法，并分析算法的正确性和计算复杂性。

★编程任务：

对于给定的 n 个程序存放在磁带上的长度和读取概率，编程计算 n 个程序的最优存储方案。

★数据输入：

由文件 input. txt 给出输入数据。第 1 行是正整数 n，表示文件个数。接下来的 n 行中，每行有 2 个正整数 a 和 b，分别表示程序存放在磁带上的长度和读取概率。实际上第 k 个程序的读取概率为 $a_k/\sum_{i=1}^{n} a_i$。对所有输入均假定 $c=1$。

课内练习16.1

2、磁盘文件最优存储问题

★问题描述：

设磁盘上有 n 个文件 $f_1, f_2, \cdots, f_n$，每个文件占用磁盘上的 1 个磁道。这 n 个文件的检索概率分别是 $p_1, p_2, \cdots, p_n$，且 $\sum_{i=1}^{n} p_i = 1$。磁头从当前磁道移到被检信息磁道所需的时间可用这 2 个磁道之间的径向距离来度量。如果文件 f_i 存放在第 i 道上，$1 \leqslant i \leqslant n$，则检索这 n 个文件的期望时间是 $\sum_{1 \leqslant i < j \leqslant n} p_i p_j d(i,j)$。其中 $d(i,j)$ 是第 i 道与第 j 道之间的径向距离 $|i-j|$。

磁盘文件的最优存储问题要求确定这 n 个文件在磁盘上的存储位置，使期望检索时间达到最小。试设计一个解此问题的算法，并分析算法的正确性与计算复杂性。

★编程任务：

对于给定的文件检索概率，编程计算磁盘文件的最优存储方案。

★数据输入：

由文件 input. txt 给出输入数据。第 1 行是正整数 n，表示文件个数。第 2 行有 n 个正整数 a_i，表示文件的检索概率。实际上第 k 个文件的检索概率应为 $a_k / \sum_{i=1}^{n} a_i$。

★结果输出：

将编程计算出的最小期望检索时间输出到文件 output. txt。

输入文件示例	输出文件示例
input. txt	output. txt
5	0. 547396
33 55 22 11 9	